МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Департамент цифровых, робототехнических систем и электроники

«Основы работы с библиотекой NumPy»
Отчет по практической работе № 2
по дисциплине «ML и искусственный интеллект»
Вариант 8

Выполнил студент группы ИВТ-б-о-24-1
Каиров Вадим Сосланович
«__» сентября 2025г.
Подпись студента_____
Работа защищена « »_____20__г.
Проверил Воронкин Р.А. _____
(подпись)

Ставрополь 2026

Цель работы: исследовать базовые возможности библиотеки NumPy языка программирования Python.

Ссылка на ГитХаб: https://github.com/ozetin/ML_laba2

Конспект теоретического материала:

Что такое библиотека NumPy?

NumPy (от "Numerical Python") - это открытая библиотека для языка Python, предназначенная для эффективных вычислений с многомерными массивами и матрицами. NumPy предоставляет множество высокоуровневых математических функций для работы с этими структурами данных и выполнения математических операций. Это одна из самых популярных библиотек для научных и инженерных вычислений, а также неотъемлемая часть многих других библиотек Python, таких как SciPy, Pandas, и TensorFlow.

Основная структура данных в NumPy - это многомерный массив, называемый ndarray. Это объект, который представляет собой таблицу данных (например, вектор или матрицу) и позволяет выполнять операции с ними с высокой эффективностью.

· Одномерные массивы (вектора): могут быть использованы для представления наборов данных.

· Двумерные массивы (матрицы): широко применяются в линейной алгебре и машинном обучении.

· Многомерные массивы (тензоры): важны в глубоких нейронных сетях и других сложных вычислениях.

Ход работы:

В ходе работы были выполнены практические задания.

1. Создайте массив NumPy размером 3x3, содержащий числа от 1 до 9. Умножьте все элементы массива на 2, а затем замените все элементы больше 10 на 0. Выведите итоговый массив.

Решение:

```
import numpy as np

mas = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])
mas = 2 * mas
mas[mas < 10] = 0
print(mas)
```

Результат выполнения программы:

```
[[ 0  0  0]
 [ 0 10 12]
 [14 16 18]]
```

2. Создайте массив NumPy из 20 случайных целых чисел от 1 до 100. Найдите и выведите все элементы, которые делятся на 5 без остатка. Затем замените их на -1 и выведите обновленный массив.

Решение:

```
import numpy as np

mas = np.random.randint(1, 101, size = 20)
print(mas)
print(mas[mas % 5 == 0])
mas[mas % 5 == 0] = -1
print(mas)
```

Результат выполнения программы:

```
[ 2 46 72 59  1 89 42 94 74 47 65 55 39 22 92 72 65 26 83 38]
[65 55 65]
[ 2 46 72 59  1 89 42 94 74 47 -1 -1 39 22 92 72 -1 26 83 38]
```

3. Создайте два массива NumPy размером 1x5, заполненные случайными числами от 0 до 50.

- Объедините эти массивы в один двумерный массив (по строкам).
- Разделите полученный массив на два массива, каждый из которых содержит 5 элементов.

Выведите все промежуточные и итоговые результаты.

Решение:

```python
import numpy as np


mas1 = np.random.randint(0, 51, size=(1, 5))
mas2 = np.random.randint(0, 51, size=(1, 5))

print(mas1)
print(mas2)

mas = np.concatenate((mas1, mas2), axis=0)
print(mas)

split1, split2 = np.vsplit(mas, 2)

print("Первый разделенный массив")
print(split1)
print("Второй разделенный массив")
print(split2)
```

Результат выполнения программы:

```
[[10 40 14 36 34]]
[[49 42  4 33 24]]
[[10 40 14 36 34]
 [49 42  4 33 24]]
Первый разделенный массив
[[10 40 14 36 34]]
Второй разделенный массив
[[49 42  4 33 24]]
```

4. Создайте массив из 50 чисел, равномерно распределенных от -10 до 10. Вычислите сумму всех элементов, сумму положительных элементов и сумму отрицательных элементов. Выведите результаты.

Решение:

```python
import numpy as np

arr = np.linspace(-10, 10, 50)

print("Массив из 50 чисел от -10 до 10:")
print(arr)

summa = np.sum(arr)
print(f"Сумма всех элементов: {summa}")

polozh_summa = np.sum(arr[arr > 0])
print(f"Сумма положительных элементов: {polozh_summa}")

otriz_sum = np.sum(arr[arr < 0])
print(f"Сумма отрицательных элементов: {otriz_sum}")
```

Результат выполнения программы:

```
Массив из 50 чисел от -10 до 10:
[-10.          -9.59183673  -9.18367347  -8.7755102   -8.36734694
  -7.95918367  -7.55102041  -7.14285714  -6.73469388  -6.32653061
  -5.91836735  -5.51020408  -5.10204082  -4.69387755  -4.28571429
  -3.87755102  -3.46938776  -3.06122449  -2.65306122  -2.24489796
  -1.83673469  -1.42857143  -1.02040816  -0.6122449   -0.20408163
   0.20408163   0.6122449    1.02040816   1.42857143   1.83673469
   2.24489796   2.65306122   3.06122449   3.46938776   3.87755102
   4.28571429   4.69387755   5.10204082   5.51020408   5.91836735
   6.32653061   6.73469388   7.14285714   7.55102041   7.95918367
   8.36734694   8.7755102    9.18367347   9.59183673  10.        ]
Сумма всех элементов: 7.105427357601002e-15
Сумма положительных элементов: 127.55102040816328
Сумма отрицательных элементов: -127.55102040816327
```

5. Создайте:

- Единичную матрицу размером 4x4.
- Диагональную матрицу размером 4x4 с диагональными элементами [5, 10, 15, 20] (не использовать циклы).

Найдите сумму всех элементов каждой из этих матриц и сравните результаты.

Решение:

```
import numpy as np

ones_matrix = np.eye(4)
print(ones_matrix)

diagonal = [5, 10, 15, 20]
diagonal_matrix = np.diag(diagonal)
print(diagonal_matrix)

sum_ones = np.sum(ones_matrix)
sum_diagonal = np.sum(diagonal_matrix)

print(f"Сумма всех элементов единичной матрицы: {sum_ones}")
print(f"Сумма всех элементов диагональной матрицы: {sum_diagonal}")
print(f"Разница между суммами: {abs(sum_diagonal - sum_ones)}")
```

Результат выполнения программы:

```
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
[[ 5  0  0  0]
 [ 0 10  0  0]
 [ 0  0 15  0]
 [ 0  0  0 20]]
Сумма всех элементов единичной матрицы: 4.0
Сумма всех элементов диагональной матрицы: 50
Разница между суммами: 46.0
```

6. Создайте две квадратные матрицы NumPy размером 3x3, заполненные случайными целыми числами от 1 до 20. Вычислите и выведите:

- Их сумму
- Их разность
- Их поэлементное произведение

Решение:

```
import numpy as np

matrix1 = np.random.randint(1, 21, size=(3, 3))
matrix2 = np.random.randint(1, 21, size=(3, 3))

print(matrix1)
print(matrix2)

sum_matrix = matrix1 + matrix2
print("\nСумма матриц (A + B):")
print(sum_matrix)

razn_matrix = matrix1 - matrix2
print("\nРазность матриц (A - B):")
print(razn_matrix)

# Вычисляем поэлементное произведение
proizv = matrix1 * matrix2
print("\nПоэлементное произведение (A * B):")
print(proizv)
```

Результат выполнения программы:

```
[[11 15 12]
 [16  1  8]
 [ 6 17 19]]
[[10  3  2]
 [12 14  9]
 [19 18  5]]

Сумма матриц (A + B):
[[21 18 14]
 [28 15 17]
 [25 35 24]]

Разность матриц (A - B):
[[  1  12  10]
 [  4 -13  -1]
 [-13  -1  14]]

Поэлементное произведение (A * B):
[[110  45  24]
 [192  14  72]
 [114 306  95]]
```

7. Создайте две матрицы NumPy:

- Первую размером 2x3, заполненную случайными числами от 1 до 10.
- Вторую размером 3x2, заполненную случайными числами от 1 до 10.

Выполните матричное умножение (@ или np.dot) и выведите результат.

Решение:

```
import numpy as np

matrix1 = np.random.randint(1, 11, size=(2, 3))
print("Первая матрица (2×3):")
print(matrix1)

matrix2 = np.random.randint(1, 11, size=(3, 2))
print("\nВторая матрица (3×2):")
print(matrix2)

result = matrix1 @ matrix2
print("\nРезультат матричного умножения (2×3 × 3×2 = 2×2):")
print(result)
```

Результат выполнения программы:

```
Первая матрица (2×3):
[[3 3 4]
 [6 1 7]]

Вторая матрица (3×2):
[[10 10]
 [ 9  5]
 [ 4  7]]

Результат матричного умножения (2×3 × 3×2 = 2×2):
[[ 73  73]
 [ 97 114]]
```

8. Создайте случайную квадратную матрицу 3x3. Найдите и выведите:

Определитель этой матрицы

Обратную матрицу (если существует, иначе выведите сообщение, что матрица вырождена)

Используйте функции np.linalg.det и np.linalg.inv .

Решение:

```python
import numpy as np

matrix = np.random.randint(1, 11, size=(3, 3))

print("Случайная матрица 3×3:")
print(matrix)

det = np.linalg.det(matrix)

if abs(det) != 0:
    inv_matrix = np.linalg.inv(matrix)
    print("\nОбратная матрица существует:")
    print(inv_matrix)

    prover = matrix @ inv_matrix
    print("\nПроверка: A × A⁻¹ = E (единичная матрица):")
    print(np.round(prover, 2))
else:
    print("\nМатрица вырождена (определитель = 0), обратная матрица не существует!")
```

Результат выполнения программы:

```
Случайная матрица 3×3:
[[8 7 8]
 [9 1 3]
 [3 2 7]]

Обратная матрица существует:
[[-0.004  0.132 -0.052]
 [ 0.216 -0.128 -0.192]
 [-0.06  -0.02   0.22 ]]

Проверка: A × A⁻¹ = E (единичная матрица):
[[ 1.  0.  0.]
 [ 0.  1.  0.]
 [-0. -0.  1.]]
```

9. Создайте матрицу NumPy размером 4x4, содержащую случайные целые числа от 1 до 50. Выведите:

- Исходную матрицу
- Транспонированную матрицу
- След матрицы (сумму элементов на главной диагонали)

Используйте np.trace для нахождения следа.

Решение:

```python
import numpy as np

matrix = np.random.randint(1, 51, size=(4, 4))
```

```
print("Исходная матрица (4×4):")
print(matrix)

transpon_matrix = matrix.T
print("\nТранспонированная матрица:")
print(transpon_matrix)

trace = np.trace(matrix)
print(f"\nСлед матрицы (сумма элементов на главной диагонали): {trace}")
```

Результат выполнения программы:

```
Исходная матрица (4×4):
[[ 8 14 30 10]
 [28 36  3 48]
 [ 5 46 50 23]
 [14 29 36 13]]

Транспонированная матрица:
[[ 8 28  5 14]
 [14 36 46 29]
 [30  3 50 36]
 [10 48 23 13]]

След матрицы (сумма элементов на главной диагонали): 107
```

10. Решить систему линейных уравнений вида:

$$\begin{cases} 2x + 3y - z = 5 \\ 4x - y + 2z = 6 \\ -3x + 5y + 4z = -2 \end{cases}$$

Используйте матричное представление Ax = B , где A – матрица коэффициентов, x - вектор неизвестных, в - вектор правой части. Решите систему с помощью np.linalg.solve и выведите результат.

Решение:

```
import numpy as np

A = np.array([
    [2, 3, -1],
    [4, -1, 2],
    [-3, 5, 4]
])

B = np.array([5, 6, -2])

print("Система линейных уравнений:")
print("2x + 3y - z = 5")
print("4x - y + 2z = 6")
print("-3x + 5y + 4z = -2")

print("Матрица коэффициентов A:")
print(A)
```

```
print()
print("Вектор правой части B:")
print(B)
print()

resh = np.linalg.solve(A, B)

print("Решение системы:")
print(f"x = {resh[0]}")
print(f"y = {resh[1]}")
print(f"z = {resh[2]}")
```

Результат выполнения программы:

```
Система линейных уравнений:
2x + 3y - z = 5
4x - y + 2z = 6
-3x + 5y + 4z = -2
Матрица коэффициентов A:
[[ 2  3 -1]
 [ 4 -1  2]
 [-3  5  4]]

Вектор правой части B:
[ 5  6 -2]

Решение системы:
x = 1.6396396396396398
y = 0.5765765765765767
z = 0.009009009009008976
```

Индивидуальное задание:

Решите индивидуальное задание согласно варианта. Каждое задание предусматривает построение системы линейных уравнений. Решите полученную систему уравнений с использованием библиотеки NumPy. Для решения системы используйте метод Крамера и матричный метод. Сравните полученные результаты, с результатами, полученными с помощью пр. linalg.solve . Перераспределение бюджета. Городской бюджет распределяется между образованием, здравоохранением и транспортом. На здравоохранение выделяется в 1.5 раза больше, чем на транспорт, а на образование - на 20% больше, чем на здравоохранение. Всего в бюджете 5 000 000 рублей. Сколько денег выделено на каждую сферу?

Решение:

```
import numpy as np

A = np.array([[-1.5, 1, 0],
              [0, -1.2, 1],
              [1, 1, 1]])
```

```
B = np.array([0, 0, 5_000_000])

matr = np.linalg.inv(A) @ B

det_A = np.linalg.det(A)

def kram(A, B, col):
    A_i = A.copy()
    A_i[:, col] = B
    return np.linalg.det(A_i) / det_A

kramer = np.array([kram(A, B, i) for i in range(3)])

solve = np.linalg.solve(A, B)

print("Матричный метод:", matr)
print("Метод Крамера:  ", kramer)
print("np.linalg.solve: ", solve)
```

Результат выполнения программы:

```
Матричный метод: [1162790.69767442 1744186.04651163 2093023.25581395]
Метод Крамера:   [1162790.69767442 1744186.04651163 2093023.25581396]
np.linalg.solve:  [1162790.69767442 1744186.04651163 2093023.25581395]
```

Контрольные вопросы:

1. Каково назначение библиотеки NumPy?

Библиотека NumPy предназначена для эффективной работы с большими массивами чисел и предоставления высокоуровневых математических функций для операций линейной алгебры, преобразования Фурье и случайных чисел. Она служит основой для многих научных пакетов Python, обеспечивая быстрый и удобный доступ к элементам массива, операции векторизации и эффективные алгоритмы для численных расчетов.

2. Что такое массивы ndarray?

Массивы ndarray (n-dimensional array) — основной объект библиотеки NumPy, представляющий N-мерные однородные массивы фиксированного размера. Они обеспечивают эффективную обработку больших объёмов данных благодаря оптимизации на уровне C и поддержке быстрых операций с элементами массива, таких как арифметические операции, срезы, транспонирование и многие другие научные и математические функции.

3. Как осуществляется доступ к частям многомерного массива?

Доступ к частям многомерного массива в NumPy осуществляется с помощью индексирования и нарезания (slicing). Индексация производится указанием координат элемента в квадратных скобках, начиная с нуля. Нарезка задаётся диапазоном индексов в формате [start:stop:step], позволяющим извлекать подмножества элементов массива. Например, для извлечения первого столбца двумерного массива используется запись array[:, 0].

4. Как осуществляется расчет статистик по данным?

Расчёт статистики по данным в NumPy осуществляется с помощью специализированных методов объектов ndarray: mean() — среднее значение, std() — стандартное отклонение, min() и max() — минимальные и максимальные значения соответственно, sum() — сумма элементов, prod() — произведение элементов, argmin() и argmax() — индексы минимального и максимального значений. Эти методы принимают аргумент axis, определяющий ось расчёта статистики (по строкам или столбцам).

5. Как выполняется выборка данных из массивов ndarray?

Выборка данных из массивов ndarray в NumPy выполняется через индексацию и нарезку (slicing). Можно выбирать элементы по отдельным индексам или диапазонам индексов, используя квадратные скобки. Например, arr[1, 2] выбирает элемент из второй строки и третьего столбца. Для сложных условий применяется булево индексирование, например, arr[arr > threshold] возвращает элементы, превышающие заданный порог. Также поддерживается выборка с помощью целых массивов индексов, например, arr[[0, 2], :] выберет первую и третью строку массива.

6. Приведите основные виды матриц и векторов. Опишите способы их создания в языке Python?

Матрицы и векторы представляют собой структуры данных, используемые для представления двумерных и одномерных массивов соответственно. Основные виды матриц включают единичные матрицы, диагональные матрицы, симметричные матрицы, ортогональные матрицы и

треугольные матрицы. Векторы бывают скалярные (одномерные) и вектор-столбец (двумерные с одним столбцом).

```
import numpy as np

vector = np.array([1, 2, 3])

matrix = np.array([[1, 2, 3], [4, 5, 6]])

identity_matrix = np.eye(3)

diagonal_matrix = np.diag([1, 2, 3])

random_matrix = np.random.rand(3, 3)
```

7. Как выполняется транспонирование матриц?

Транспонирование матриц в NumPy выполняется методом .T объекта ndarray или функцией numpy.transpose(). Транспонированная матрица получается путём замены строк на столбцы исходной матрицы.

8. Приведите свойства операции транспонирования матриц.

Ассоциативность относительно умножения, Дистрибутивность относительно суммы, идемпотентность, умножения скаляра на матрицу.

9. Какие имеются средства в библиотеке NumPy для выполнения транспонирования матриц?

Метод .T, функция numpy.transpose(), аргумент axes.

10. Какие существуют основные действия над матрицами?

Сумма, вычитание, умножение, умножение на число, транспонирование, определитель матрицы, обращение матрицы, собственные числа и собственный векторы.

11. Как осуществляется умножение матрицы на число?

```
matrix = np.array([[1, 2],
                   [3, 4]])
```

result = matrix * 2

12. Какие свойства операции умножения матрицы на число?

Ассоциативность, дистрибутивность относительно сложения матриц, чисел.

13. Как осуществляется операции сложения и вычитания матриц?

A = np.array([[1, 2], [3, 4]])

B = np.array([[5, 6], [7, 8]])

C = A + B

D = A - B

14. Каковы свойства операций сложения и вычитания матриц?

Коммутативность, ассоциативность, нулевая матрица.

15. Какие имеются средства в библиотеке NumPy для выполнения операций сложения и вычитания матриц?

Оператор + и -, функции add() и subtract().

16. Как осуществляется операция умножения матриц?

A = np.array([[1, 2], [3, 4]])

B = np.array([[5, 6], [7, 8]])

C = np.dot(A, B)

17. Каковы свойства операции умножения матриц?

Ассоциативность, распределительность относительно сложения, не коммутативность.

18. Какие имеются средства в библиотеке NumPy для выполнения операции умножения матриц?

numpy.dot()

C = A @ B

numpy.matmul()

19. Что такое определитель матрицы?

Определитель матрицы (

Определитель матрицы - это специальное вещественное число, которое ассоциируется с каждой квадратной матрицей и описывает важные

геометрические и алгебраические характеристики матрицы, такие как изменение объёма при линейном преобразовании, наличие обратной матрицы и решение систем линейных уравнений.

20. Какие имеются средства в библиотеке NumPy для нахождения значения определителя матрицы?

linalg.det()

21. Что такое обратная матрица? Какой алгоритм нахождения обратной матрицы?

Алгоритм нахождения обратной матрицы основан на методах линейной алгебры, таких как метод Гаусса-Жордана, LU-разложение или SVD-разложение. Наиболее распространённым методом является LU-разложение (разложение матрицы на нижнюю и верхнюю треугольные матрицы), после которого находят обратную матрицу

22. Каковы свойства обратной матрицы?

Единственность, обратная к обратной, перестановочность с транспонированием.

23. Какие имеются средства в библиотеке NumPy для нахождения обратной матрицы?

A = np.array([[1, 2], [3, 4]])

inverse_A = np.linalg.inv(A)

24. Изучить метод Крамера для решения СЛУ. Привести алгоритм решения СЛУ Крамером средствами библиотеки NumPy.

1) Найти определитель инсходной матрицы A.

2) Если det(A) != 0, система имеет единственной решение.

3) Подставляя вектор B на место каждого столбцы матрицы A, вычисляются вспомогательные определитель del(i)

4) X_i = deli/del, i = 1…n

Del – определитель исходной матрицы, deli – определитель матрицы, полученной заменой i-го столбцы на вектор B

25. Изучить матричный метод для решения СЛУ. Привести алгоритм решения СЛУ матричным методом средствами библиотеки NumPy.

Метод заключается в приведении системы к виду AX = B, где A – матрица коэффициентов, X – вектор неизвестных, B – вектор правых частей. Суть метода:

1) Проверяется условие разрешимости системы(матрица A должно быть невырожденной, т.е. иметь ненулевой определитель).
2) Находится обратная матрица
3) Искомое решение находится: X = A^(-1)B

Вывод: В ходе выполнения работы были исследованы базовые возможности библиотеки NumPy, необходимые для решения систем линейных уравнений, включая создание многомерных массивов с помощью np.array(), вычисление определителя матрицы через np.linalg.det(), нахождение обратной матрицы посредством np.linalg.inv(), матричное умножение оператором @, прямое решение систем линейных уравнений функцией np.linalg.solve(), а также реализацию метода Крамера путём замены столбцов матрицы. В результате все три способа решения системы (матричный метод, метод Крамера и встроенная функция np.linalg.solve()) дали идентичные результаты с высокой точностью, что подтверждает корректность работы NumPy и эффективность его использования для задач линейной алгебры.